CHAS Security Assessment Report

Final Project: System/Application Security Assessment

System assessed: CHAS (Child Health Assessment System), version 0.7.0

Submitted: October 19, 2026

Members:

- Daren De Lumen
- Angela Ordoña
- Katherine Tating
- Angelica Joy Lanzanida

---

1. Introduction and System Description

For this project we assessed our own system, CHAS (Child Health Assessment System). We built CHAS for the school clinic of Detailen Elementary School. The nurse uses it to record each learner's weight and height, and the app computes the nutritional status and prints the forms the school needs, like the SF8 and the invitation slips for parents.

Since the app keeps health records of children, we treated privacy as the main concern. Under the Data Privacy Act of 2012 (RA 10173), health information is sensitive personal information, so the school is expected to protect it properly.

Part	What we used
Desktop app	Electron 33, packaged as a Windows installer using electron-builder
User interface	React 18 with Vite and Tailwind CSS
Server (API)	Node.js and Express, with Zod for input checking, helmet for security headers and express-rate-limit
Database	PostgreSQL 17 (embedded in the installer), accessed through Prisma
Passwords	Hashed with Argon2id
Hosting	No hosting. Everything runs offline on one Windows computer, and the server only listens on 127.0.0.1

CHAS is not a website. It has no domain and no online server. When the user opens the app, the installer's built-in server and database start on the same computer. There are only two kinds of users: the school head, who acts as the administrator, and the school nurse.

We ran all the tests on October 19, 2026 using version 0.7.0. To be safe, we did not use any real learner records. We started a separate test copy of the app that only had the 25 sample learners.

2. Testing Methodology and Tools Used

We tested on the laptop where we develop CHAS (Windows 11). We ran the built version of 0.7.0 with its own empty data folder, so the only records inside were the sample learners. Every attack in this report was aimed at that test copy, which we own.

Our approach had four parts. First we checked what the app exposes to the network. Then we scanned the source code and the libraries it uses. After that we attacked the running app, and lastly we looked at how the data is saved on the disk.

Tools we used

Tool (version)	Type of test	What we used it for
PowerShell (Get-NetTCPConnection) and a TCP connect test	Network exposure	To list the ports CHAS opens and see if they can be reached from the network. We used this in place of Nmap
npm audit (npm 10)	Third-party libraries	To find packages with known vulnerabilities in the app and the server
Semgrep OSS 1.136.0	Source code (SAST)	To look for insecure code and hard-coded secrets, using the javascript, typescript, react, secrets and owasp-top-ten rules
Electronegativity 1.10.3	Electron settings	To check the desktop window settings, navigation, preload, CSP and permissions
curl 8	Manual web test	To read the security headers, cookie flags and CORS behavior

Tool (version)	Type of test	What we used it for
sqlmap 1.10.10	SQL injection	To test the API endpoints for SQL injection while signed in
OWASP ZAP 2.16.1	Web app scanner	To crawl the app and run passive and active scans with a signed-in session
VirusTotal	File reputation	To scan the installer, CHAS Setup 0.7.0.exe, for malware
Manual testing	Login and storage	To check the default accounts, the login limit, sessions and how data is stored

Tools from the list that did not apply

Many of the suggested tools need a public website, a domain name or a public IP address. CHAS has none of these. It only runs on 127.0.0.1 inside one computer, so online scanners have nothing they can reach.

Listed tool	Why we could not use it	What we used instead
SynapsInt, SimplyWhois	They need a public domain or IP to look up	Nothing needed, since CHAS has no public footprint
C99 Subdomain Finder	Needs a registered domain	Nothing needed
MXToolbox SuperTool	Checks DNS and mail records, but CHAS does not send email	Nothing needed
Test-IPv6, IPLocation	These test the internet connection of whoever opens the site, not CHAS	Our own port and connection test
urlscan.io, ScamAdviser, Web-Check, WAF Checker, Pentest-Tools, Strix	They scan over the internet and cannot reach 127.0.0.1	OWASP ZAP and sqlmap, which run locally
Burp Suite Community	It would work, but ZAP does the same job and can run from the command line	OWASP ZAP

Listed tool	Why we could not use it	What we used instead
Nmap	The Windows version needs the Npcap driver, which needs an administrator install to scan the same computer	PowerShell port listing and connect test
testssl.sh, sslyze	CHAS uses plain HTTP inside the computer, so there is no TLS to test	Nothing; we listed this as a finding instead

3. Security Assessment Results and Evidence

Below are the results of each test. For each one we explain why we ran it, show the actual output (shortened to the important lines), and then give our findings, our interpretation and what we recommend. The complete output files are in the evidence folder that comes with this report.

3.1 Network exposure (PowerShell port listing and connect test)

We wanted to know which ports CHAS opens and whether another computer on the same Wi-Fi could connect to them. While the app was running, we listed the listening ports and then tried connecting to them using the laptop's network address (192.168.1.13).

```
LocalAddress  Port Process    PID
127.0.0.1    47810 CHAS     17332
127.0.0.1    54329 postgres  6116
::1          54329 postgres  6116
::            5432 postgres  6644   <- not CHAS (separately installed
PostgreSQL)
0.0.0.0       5432 postgres  6644   <- not CHAS

connect 127.0.0.1:47810    -> OPEN
connect 192.168.1.13:47810 -> closed/refused
connect 127.0.0.1:54329    -> OPEN
connect 192.168.1.13:54329 -> closed/refused
```

Findings: The CHAS server (port 47810) and its database (port 54329) only listen on 127.0.0.1. When we tried to connect using the network address, both refused. We also noticed another PostgreSQL server on port 5432 that listens on all interfaces. That one is not part of CHAS. It is a separate PostgreSQL installation on this laptop.

Interpretation: Since CHAS cannot be reached from the school network or the internet, most remote attacks are already ruled out. The extra PostgreSQL is outside our system, but it is still open to anyone on the same Wi-Fi, so it is worth fixing on any computer that runs CHAS.

Recommendation: Keep CHAS bound to 127.0.0.1. On school computers, stop the extra PostgreSQL service or block port 5432 in the firewall.

3.2 Third-party libraries (npm audit)

CHAS depends on many npm packages, so we ran npm audit on both the desktop app and the server to see if any of them have published vulnerabilities.

```
Desktop app (UI + build tools): 23 vulnerabilities (11 moderate, 10 high, 2
critical)
  shipped in the app:  electron 33.4.11 (high), react-router / react-router-
dom (moderate)
  build tools only:    vite, esbuild, concurrently, shell-quote, electron-
builder, undici, ...
Server:                  7 vulnerabilities (3 moderate, 3 high, 1 critical)
  proxy-addr (critical), prisma + @prisma/config + deepmerge-ts (high),
  express + body-parser + qs (moderate)
```

Findings: The total looked scary at first (30 vulnerable packages), so we went through them one by one. Most of the desktop app's results are in build tools like Vite and electron-builder, which we only use when building the installer. The ones that actually ship are Electron 33, which is already out of support and has high-severity advisories (one of them is an ASAR integrity bypass), and the react-router packages. On the server side, proxy-addr has a critical advisory and the Prisma tool chain has high ones.

Interpretation: The Electron result matters most because every user runs that code. The proxy-addr problem only affects apps that trust a reverse proxy, which CHAS does not do, and our server only answers on 127.0.0.1, so its real risk to us is low. The build tool issues affect our development laptop, not the schools.

Recommendation: Upgrade Electron to a supported version (44.x), run `npm audit fix` on both the app and the server, and test again. We also plan to make npm audit part of every release.

3.3 Source code (Semgrep OSS)

We used Semgrep to scan our own code for insecure patterns and secrets that might have been left in the code. Two of the rule packs we first tried (p/express and p/nodejsscan) would not load, so we used the javascript, typescript, react, secrets and owasp-top-ten packs.

```
Scanning 95 files with the javascript, typescript, react, secrets and owasp-
top-ten rule sets
Ran 116 rules on 95 files: 0 findings.
```

Findings: Semgrep did not find anything.

Interpretation: None of the rules matched any injection points, unsafe HTML rendering, weak crypto or saved keys in our code. We know this only covers what the free rules can detect, so a clean result does not prove the code has no flaws.

Recommendation: Keep running Semgrep before every release.

3.4 Electron configuration (Electronegativity)

Because CHAS is an Electron app, we also checked the desktop window settings with Electronegativity. Our first run only gave a few general warnings because the tool skipped our main process files, which use the .cjs extension. We copied them as .js files and ran it again with the correct Electron version (33.4.11).

```
CSP_GLOBAL_CHECK                          MEDIUM  No CSP has been detected
PERMISSION_REQUEST_HANDLER_GLOBAL_CHECK  MEDIUM  Missing
PermissionRequestHandler
AUXCLICK_JS_CHECK                         MEDIUM  main.js:103, main.js:174
PRELOAD_JS_CHECK                          MEDIUM  main.js:116  Review the use
of preload scripts
AVAILABLE_SECURITY_FIXES_GLOBAL_CHECK    INFO    Unknown Electron release
33.4.x
```

Findings: We checked each warning against our code. Only one turned out to be a real gap: we never set a permission request handler. The CSP warning is a false positive because our CSP is sent by the server as an HTTP header (see 3.5), which a code scan cannot see. The middle-click warning is already covered, since `setWindowOpenHandler` blocks every new window. For the preload, it only exposes eight named functions, and the window runs with `contextIsolation`, `sandbox` and without `nodeIntegration`.

Interpretation: Overall, the window follows Electron's main security recommendations. Adding a permission handler would stop camera, microphone or notification requests in case a page ever gets injected.

Recommendation: Add a `session.setPermissionRequestHandler` that denies all permission requests.

3.5 HTTP headers, cookies and CORS (curl)

Even if the app only runs locally, the window still talks to the server over HTTP, so we checked the headers and cookie settings with curl. We also sent a request pretending to come from another website (evil.example) to see if the server would accept it.

```
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-
src 'none'; script-src 'self'; ...
X-Frame-Options: SAMEORIGIN    X-Content-Type-Options: nosniff    Referrer-
Policy: no-referrer
Cache-Control: no-store        RateLimit-Policy: 300;w=60
Set-Cookie: chas_session=OVaZu2...(redacted); Expires=+12 h; HttpOnly;
Secure; SameSite=Lax
OPTIONS /api/auth/login from Origin http://evil.example -> HTTP 403 Forbidden
```

Findings: Every response has a strict CSP and the usual security headers. The session cookie is HttpOnly, Secure and SameSite=Lax, and it lasts 12 hours. The request from the fake website was refused. The connection uses plain HTTP, so the HSTS header it sends does not really do anything.

Interpretation: XSS, clickjacking and CSRF are well covered by these settings. Plain HTTP is only acceptable here because the traffic never leaves the computer. One thing we realized is that 12 hours is long for a shared computer in a school clinic, where someone might forget to sign out.

Recommendation: Keep the current headers, and add an automatic sign-out after around 15 minutes of no activity.

3.6 SQL injection (sqlmap)

We used sqlmap to check if any API parameter could be used for SQL injection. We signed in as the head and gave sqlmap the session cookie so it could reach the protected endpoints. We also added a short delay between requests so we would not hit the app's rate limit.

```
GET /api/learners/<id>*     URI parameter '#1*' does not seem to be
injectable
GET /api/activity?limit=5   GET parameter 'limit' does not seem to be
injectable
                            all tested parameters do not appear to be
injectable
```

Findings: sqlmap did not find any injectable parameter (level 3, risk 1). When we looked at the code, all database access goes through Prisma, and the only raw query is a fixed `SELECT 1` health check with no user input.

Interpretation: Using parameterized queries through Prisma protects CHAS from SQL injection. We were not able to run sqlmap against the login form, because the rate limit (see 3.9) blocks automated guessing after ten wrong attempts. In a way that also showed that the protection works.

Recommendation: Keep all database access in Prisma and never use `$queryRawUnsafe`.

3.7 Web application scan (OWASP ZAP 2.16.1)

ZAP was our main web scanner. Getting it to run took a few tries. In command-line mode it crashed at start, so we switched to daemon mode and changed Java's temporary folder before the network part would load. We signed in as the head and gave ZAP the session cookie, then fed it 13 API addresses because its crawler cannot follow the app's in-page routes by itself. It crawled 30 URLs and ran the active scan for 5 minutes and 42 seconds, using one thread with a 250 ms delay so the rate limit would not block it.

OWASP ZAP scan - CHAS 0.7.0 (test copy)

Site: http://127.0.0.1:47810

Generated on Mon, 19 Oct 2026 12:58:53

ZAP Version: 2.16.1

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	0
Informational	3
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
CSP: Wildcard Directive	Medium	2
CSP: style-src unsafe-inline	Medium	2
Information Disclosure - Suspicious Comments	Informational	1
Modern Web Application	Informational	2
User Agent Fuzzer	Informational	24

Alert Detail

Medium	CSP: Wildcard Directive
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://127.0.0.1:47810
Method	GET
Parameter	Content-Security-Policy
Attack	
Evidence	default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'none';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' 'unsafe-inline';upgrade-insecure-requests;connect-src 'self'
Other Info	The following directives either allow wildcard sources (or ancestors), are not defined, or are overly broadly defined: font-src
URL	http://127.0.0.1:47810/
Method	GET
Parameter	Content-Security-Policy
Attack	
Evidence	default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'none';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' 'unsafe-inline';upgrade-insecure-requests;connect-src 'self'
Other Info	The following directives either allow wildcard sources (or ancestors), are not defined, or are overly broadly defined: font-src
Instances	2
Solution	Ensure that your web server, application server, load balancer, etc. is properly configured to set the Content-Security-Policy header.
Reference	https://www.w3.org/TR/CSP/ https://caniuse.com/#search=content+security+policy https://content-security-policy.com/ https://github.com/HtmlUnit/htmlunit-csp https://developers.google.com/web/fundamentals/security/csp#policy_applies_to_a_wide_variety_of_resources
CWE Id	693
WASC Id	15
Plugin Id	10055

Medium	CSP: style-src unsafe-inline

Findings: ZAP reported 0 High, 2 Medium, 0 Low and 3 Informational alerts.

ZAP alert	ZAP risk	Instances	What it means for CHAS
CSP: Wildcard Directive	Medium	2	`font-src` allows `https:`, so fonts could be loaded from any HTTPS site
CSP: style-src unsafe-inline	Medium	2	Inline styles are allowed, which makes the CSP weaker against CSS injection
Information Disclosure - Suspicious Comments	Informational	1	The word “select” appears in the bundled React library; nothing secret is exposed
Modern Web Application	Informational	2	Just notes that CHAS is a JavaScript single-page app
User Agent Fuzzer	Informational	24	Responses stayed the same for different browsers; not an issue

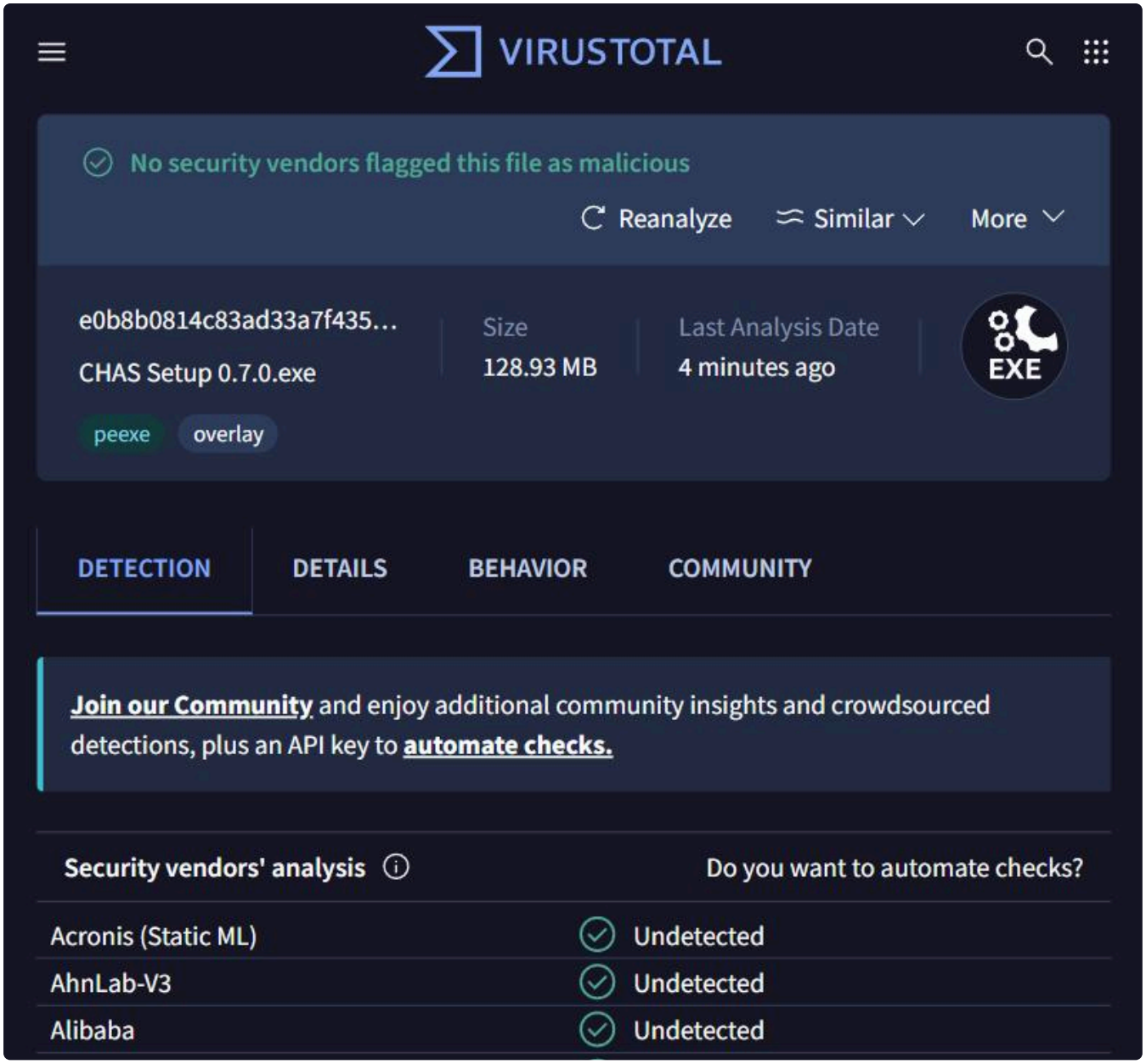
Interpretation: We were relieved that the active scan rules for SQL injection, XSS, path traversal and command injection did not raise anything. The two medium alerts are about how strict our CSP is. They are not a hole someone can directly use, since CHAS does not load fonts from the internet and is usually offline. For our system we rated both as Low.

Recommendation: Change `font-src` to `'self' data:` and move the inline styles into style sheets so that `'unsafe-inline'` can be removed.

3.8 Installer reputation (VirusTotal)

We wanted to make sure the installer that the school will download is clean. When we first searched for its SHA-256 on VirusTotal, the site said “Item not found” because nobody had uploaded it before. So we uploaded the file ourselves.

```
File:     CHAS Setup 0.7.0.exe (128.93 MB)
SHA-256:  e0b8b0814c83ad33a7f43592fa67aea6ac552a58b84a6cb98531c83cc40811a2
Result:   No security vendors flagged this file as malicious
Engines:  73 listed - 0 malicious, 0 suspicious, 57 undetected,
          10 timed out, 6 unable to process the file type
```



Findings: None of the antivirus engines flagged the installer as malicious or suspicious. Ten engines timed out, probably because the file is almost 129 MB, and six could not open the installer format, so those gave no result.

Interpretation: The installer has no known malware. A clean result only means no engine recognized a threat. It does not say anything about bugs inside the app. Also, our installer is not signed, so Windows SmartScreen may still show a warning when someone installs it.

Recommendation: Sign the installer with a code-signing certificate, share the SHA-256 together with each download, and scan every new release on VirusTotal before giving it to the school.

3.9 Authentication and access control (manual tests)

For this part we tested the login by hand using curl. We tried the default accounts, typed the wrong password many times in a row, and signed in as the nurse to see if that account could reach the head's functions.

```
POST /api/auth/sign-in admin / admin12345678 (published default)  -> HTTP 200
signed in
12 wrong passwords for admin:  attempts 1-10 -> 401,  attempts 11-12 -> 429
the correct password right after -> 429 "Too many sign-in attempts. Wait
fifteen minutes"

nurse account:  GET /api/users -> 403   POST /api/users -> 403   PUT
/api/school -> 403
                POST /api/school-year/rollover -> 403   GET /api/learners ->
200
no session:     GET /api/users -> 401   forged session token -> 401
```

Findings: The head and nurse accounts come with default passwords that are written in our own manual, and CHAS does not ask the user to change them on the first login. After ten wrong passwords, the app blocked all logins from our address for 15 minutes. We actually locked ourselves out during this test and had to wait before we could continue. The nurse got 403 on every head-only function, and requests without a session or with a fake token got 401. A wrong username and a wrong password return the same message and take about the same time, so someone cannot use the login to guess which usernames exist.

Interpretation: The default passwords are the easiest way in. Anyone who has read the manual can sign in as the head on a computer where the password was never changed. The good news is that the role checks are done on the server, not just hidden in the interface.

Recommendation: Require a new password the first time each default account signs in.

3.10 Data at rest (manual inspection)

Lastly we looked at the data folder directly, to see what someone could read if they copied the files or got hold of a backup.

```
Sample learner surname 'Alvarez' found as plain text in:
  <data>/pgdata/base/16384/16454
  <data>/pgdata/base/16384/16523
settings.json (beside the database):  pgPassword = 2c13...(48 chars),
sessionSecret = 61f8...(64 chars)
folder access: SYSTEM, Administrators, the Windows user (no other users)
PostgreSQL pg_hba.conf: host all all 127.0.0.1/32 password
```

Findings: We were able to find a sample learner's surname just by searching the raw database files. The database files, the automatic backups and the USB backups are all saved without encryption. The database password and the session secret are stored in plain text in settings.json, right beside the database. Windows only lets the owner and administrators open the folder. We also saw that the database uses the older `password` login method instead of `scram-sha-256`.

Interpretation: For a school, this is the most serious finding. If a USB backup gets lost, or the laptop is stolen and the disk is not encrypted, all the learners' health records can be read. The other protections in the app do not help once the files are outside the computer, and RA 10173 requires the school to protect this kind of data.

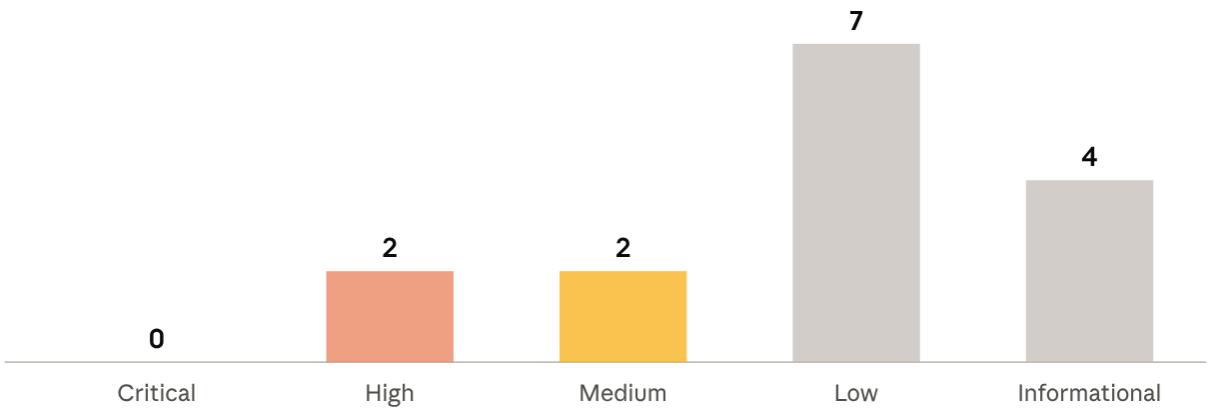
Recommendation: Turn on BitLocker or Windows device encryption on every computer that runs CHAS, add password-protected encryption to the backups before they are copied to USB, and switch the database to `scram-sha-256`.

4. Analytics, Charts and Interpretation

All in all we recorded 15 findings: 0 Critical, 2 High, 2 Medium, 7 Low and 4 Informational. We rated the severity ourselves based on how CHAS is actually used (offline, on one computer, reachable only from 127.0.0.1). Because of this, some of our ratings are lower or higher than what the tools reported.

Findings by severity: none Critical, two High

15 findings in CHAS 0.7.0, severity rated by the team for this offline, single-computer system

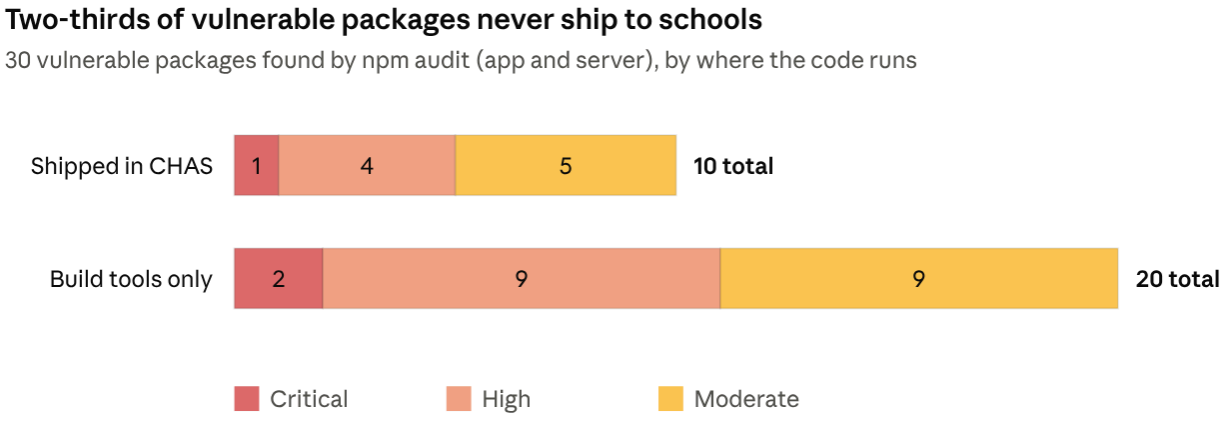


Based on our findings register below. Tests done on October 19, 2026 on the CHAS 0.7.0 test copy.

Findings register

#	Finding	Severity	Found by	Status
F1	Learner health records are not encrypted in the database files and in all backups	High	Manual (3.10)	Open
F2	The Electron 33 runtime is out of support and has published high-severity advisories	High	npm audit (3.2)	Open
F3	The default head and nurse passwords are published and users are not forced to change them	Medium	Manual (3.9)	Open
F4	Server libraries with advisories: proxy-addr (critical), Prisma tool chain (high), express/qs (moderate)	Medium	npm audit (3.2)	Open
F5	The database password and session secret are saved in plain text in settings.json and the backups	Low	Manual (3.10)	Open
F6	The database uses the `password` method instead of `scram-sha-256`	Low	Manual (3.10)	Open

#	Finding	Severity	Found by	Status
F7	The CSP `font-src` allows any HTTPS source	Low	OWASP ZAP (3.7)	Open
F8	The CSP `style-src` allows `'unsafe-inline'`	Low	OWASP ZAP (3.7)	Open
F9	No Electron permission request handler	Low	Electronegativity (3.4)	Open
F10	No automatic sign-out; a session lasts 12 hours on shared computers	Low	Manual (3.5)	Open
F11	react-router open-redirect advisory in the shipped interface	Low	npm audit (3.2)	Open
F12	Plain HTTP between the window and the server (only inside the computer); the HSTS header has no effect	Informational	curl (3.5)	Accepted
F13	20 advisories in build tools that are not included in the installer	Informational	npm audit (3.2)	Open
F14	A separate PostgreSQL service on the laptop listens on all interfaces (port 5432)	Informational	Port test (3.1)	Outside CHAS
F15	Suspicious comment in the bundled React library	Informational	OWASP ZAP (3.7)	Accepted



From npm audit on October 19, 2026: desktop app (23 packages) and server (7 packages). We counted the server packages as shipped.

Only a third of the vulnerable packages actually reach the school's computer. The catch is that those ten include Electron itself, so if we upgrade Electron and the server libraries in one release, most of the shipped risk goes away.

What passed

Twelve of the protections we tested held up: the ports are only open on 127.0.0.1, the CSP and security headers are strict, requests from other websites are refused, the session cookie is HttpOnly, Secure and SameSite, password guessing is limited, usernames cannot be guessed from the login, roles are checked on the server, sqlmap found no SQL injection, ZAP found no XSS or injection, Semgrep came back clean, the Electron window is isolated (`contextIsolation`, `sandbox`, no new windows or outside navigation), and passwords are hashed with Argon2id.

Our most important findings

1. **F1, unencrypted records.** If a USB backup gets lost or a laptop without disk encryption is stolen, every learner's health data can be read. None of the app's other protections help at that point.
2. **F2, outdated Electron.** Every user runs this code, and an unsupported version will not get security fixes anymore.
3. **F3, default passwords.** Anyone who has seen the manual can sign in as the head on a computer where the password was never changed.

Unresolved issues and limitations

- We did not fix anything during the assessment, so all 15 findings are still open and this report describes version 0.7.0 as it was released.
- We only tested a copy with 25 sample learners on our own laptop, not a real school computer or real data.
- We did not use Nmap or Burp Suite. Nmap needs Npcap, which needs an administrator install, and ZAP already covered what we would have used Burp for.
- ZAP's crawler could not follow the app's in-page routes, so we had to give it 13 API addresses ourselves. We did not fully fuzz the request bodies of every POST and PUT request.
- We could not run sqlmap on the login form because the rate limit blocked it after ten attempts.
- We only used the free Semgrep rules. Physical security, social engineering and Windows hardening were not part of our scope.
- A clean scan result does not mean the app has no flaws.

5. Recommendations and Conclusion

If we make the four changes below, both High and both Medium findings will be closed. We listed them in the order we plan to do them.

Priority	Fix	Findings closed	Estimated effort
1	Turn on BitLocker or Windows device encryption on every CHAS computer, and encrypt backups with a password before they are copied to USB or the cloud	F1, F5	Disk encryption is quick to turn on; encrypted backups will take us about a week
2	Make the default head and nurse accounts change their password on first sign-in	F3	1 to 2 days
3	Upgrade Electron to a supported version (44.x), run `npm audit fix` on the app and the server, then test everything again	F2, F4, F11, F13	2 to 3 days including testing
4	Add an automatic sign-out after 15 minutes, a permission handler that denies everything, `scram-sha-256` for the database, and a stricter CSP (`font-src 'self' data:` and no `'unsafe-inline'`)	F6 to F10	1 to 2 days

After this, we want to run npm audit, Semgrep and a ZAP scan before every new release and keep the results together with the release notes.

Conclusion. Doing this assessment showed us that CHAS is already well protected against the most common attacks. It cannot be reached from the network, it resisted SQL injection and XSS, the roles are checked on the server, and password guessing is limited. The bigger risks turned out to be outside the running app: the unencrypted files and backups, the old Electron version and the default passwords. Once we fix these four areas, we believe CHAS will be ready to handle the health records of the learners in a way that follows RA 10173.

6. References

1. OWASP Foundation. *OWASP Top 10:2021*. owasp.org/Top10
2. OWASP Foundation. *Zed Attack Proxy (ZAP)*, version 2.16.1. zaproxy.org
3. sqlmap project. *sqlmap: automatic SQL injection tool*. sqlmap.org
4. Semgrep, Inc. *Semgrep Community Edition*, version 1.136.0. semgrep.dev
5. Doyensec. *Electronegativity*, version 1.10.3. github.com/doyensec/electronegativity
6. npm, Inc. *npm audit* documentation. docs.npmjs.com/cli/commands/npm-audit
7. VirusTotal. *File analysis service*. virustotal.com
8. Electron. *Security checklist* and *Electron release timelines*. electronjs.org/docs/latest/tutorial/security
9. GitHub Advisory Database (advisories cited by npm audit). github.com/advisories
10. PostgreSQL Global Development Group. *Client authentication: password methods*. postgresql.org/docs/17/auth-password.html
11. Republic of the Philippines. *Data Privacy Act of 2012 (RA 10173)*. National Privacy Commission. privacy.gov.ph/data-privacy-act